



VRIJE
UNIVERSITEIT
BRUSSEL



Bachelor thesis
Bachelor of Science in Engineering Technology:
Electronics-ICT - networks

IOT FOR SOUND POLLUTION

Sound Level Meter

Louka Michaël Grignard

2022-2023

promoter: Abdellah Touhafi

ENGINEERING SCIENCES

Acknowledgements

I thank my promoter and the members of the jury who pointed me in the right direction during the evaluation moments.

Louka Grignard

This bachelor thesis will cover the process and logic behind the designing of an IoT node able to measure sound. Together with another bachelor thesis, which covers how to build LoRaWAN networks, this research can be used to deploy a sound pollution monitoring network. Noise pollution is a growing problem in our society. Currently there is insufficient data available to create a good policy to tackle this problem.

We will give a detailed explanation on why we chose certain components and how these components put constraints on each other. A big part of this thesis will also go over all the theory behind sound and how to measure it.

Another important part of this project is the SPI communication protocol which allows the development board to communicate with the ADC. The exact working of this will also be explained in this thesis.

Finally we will discuss how to correctly convert the measured data to values that humans can easily understand. This should help the user to interpret the data correctly and thus create a correct action plan.

Contents

Abstract	ii
Contents	iii
List of Figures	v
List of Tables	vi
Listings	vii
Nomenclature	viii
1 Introduction	1
1.1 Context	1
1.2 Objectives	1
1.3 Structure of the manuscript	2
2 Background	3
2.1 Internet of Things	3
2.2 Sound	3
2.2.1 Characteristics of sound	3
2.2.1.1 Sensitivity	4
2.2.1.2 Noise	4
2.2.1.3 Signal-To-Noise Ratio	4
2.2.1.4 Acoustic Overload Point	4
2.2.1.5 Equivalent Input Noise	4
2.2.1.6 Dynamic Range	5
2.2.1.7 Frequency Response	5
2.2.1.8 Power Supply Rejection	5
2.2.1.9 Directionality	6
2.2.2 MEMs vs ECM microphone	7
2.2.3 Sampling	8
2.3 SPI protocol	10
2.4 LoRa and LoRaWAN	12
2.5 DFT	12
3 Hardware	14
3.1 Development Board	14
3.2 Components	15
3.2.1 Microphone	15
3.2.2 ADC	16
3.2.3 Low Pass Filter	16
3.3 PCB	17

4	Software	22
4.1	Micropython	22
4.2	code	22
4.2.1	boot.py	22
4.2.2	main.py	27
4.3	Data Processing	28
4.3.1	Calculating the SPL in dB	28
4.3.2	Calculating the DFT	29
4.3.3	Calibration	30
5	Future work	32
	Bibliography	33

List of Figures

2.1	sound spectrum	3
2.2	Relation between dB SPL input and dBV output for the ICS-40300	5
2.3	Frequency response unidirectional microphone	6
2.4	Frequency response omnidirectional microphone	6
2.5	ECM built, source: <i>MEMS versus ECM</i> 2023	7
2.6	MEMs built, source: <i>MEMS versus ECM</i> 2023	7
2.7	Aliasing example	8
2.8	Aliasing example with higher sampling rate	8
2.9	Frequency spectrum of S_c	9
2.10	Frequency spectrum of discrete signal S_d	9
2.11	Frequency spectrum of discrete signal sampled at sub Nyquist frequency	10
2.12	SPI communication between 2 nodes	10
2.13	SPI daisy-chain setup	11
2.14	SPI multinode setup	12
2.15	Fourier Decomposition, adepted from Kordahl, 2021	13
3.1	schematic of the microphone	17
3.2	schematic of the ADC	18
3.3	schematic of the connectors	19
3.4	2D front view	20
3.5	2D back view	20
3.6	3D front view	21
3.7	3D back view	21
4.1	command byte structure, source ‘MCP3461’ 2020.	23
4.2	command byte	24
4.3	Calibration setup	30

	List of Tables
3.1 Comparison development boards: LoPy4 vs FiPy	14
3.2 Comparison development boards: LoPy4 vs Raspberry Pi Pico	15

	Listings
4.1 Boot.py	22
4.2 Python function to initialize everything required for SPI communication	23
4.3 write_config function	24
4.4 Main.py file	27
4.5 DFT method	29
4.6 ulab DFT method	30

Acronyms

LoRaWAN	LoRa Wide Area Network
IoT	Internet of Things
MAC	Medium Access Control
PCB	Printed Circuit Board
IC	Integrated Circuit
ADC	Analog-digital-converter
DAC	Digital-Analog-Converter
JFET	Junction-gate-Field-Effect Transistor
LPF	Low Pass Filter
LED	Light Emitting Diode
RAM	Random Access Memory
DC	Direct Current
AC	Alternating Current
RMS	Root Mean Squared
SPI	Serial Peripheral Interface
CS	Chip Select
SCLK	Serial Clock
MOSI	Master Out Slave In
MISO	Master In Slave Out
SDI	Slave Device In
SDO	Slave Device Out
CPOL	Clock Polarity
CPHA	Clock Phase
DMCLK	Digital Master Clock
AMCLK	Analog Master Clock
CRC	Cyclic Redundancy Check
POR	Power On Reset
OSR	Over Sampling Ratio
DFT	Discrete Fourier Transform
FFT	Fast Fourier Transform
FT	Fourier Transform
SPL	Sound Pressure Level
AOP	Acoustic Overload Point
EIN	Equivalent Input Noise
DNR	Dynamic Range
SNR	Sound Noise Ratio
PSR	Power Supply Rejection
ECM	Electret Condenser Microphone

MEMs Micro-Electro-Mechanical systems

test

1.1 CONTEXT

The goal of this project is to build an IoT network that is able to measure sound pollution in cities. The project was further divided into a part that focusses on building a LoRaWAN network and a part that focusses on designing an IoT node able to measure sound. This bachelor thesis will focus on the second one.

Lets start of by looking at why we should care about sound pollution. While it may sound a bit weird, The European Environment Agency has reported(Peris, 2020) that noise pollution has a negative effect on about 20% of the european population. This translates to more then 100 million people who are negatively affected. It is long term exposure to noise that can negatively affect your health by causing annoyance, sleep disturbance, your cardiovascular metabolic systems and even cause cognitive impairment in children. According to the World Health Association(WHO) noise is the second largest environmental cause of health problems.

Currently there is not enough data to be able to create useful noise maps and action maps. That is where our project comes in. By creating an IoT network that can be placed anywhere in a city while capturing the sound and transmitting it to a central gateway we will be able to fill in the missing data and create accurate noise maps which in turn will allow for action maps to be made.

1.2 OBJECTIVES

As stated previously this project is separated into 2 thesises. The first one will focus on building an IoT network with LoRaWAN. This thesis will focus on building the node that captures and transmits data. We are interested in how loud the ambient sound is at any given time. There are 2 things we are interested in, the dB value to know how loud the sound is and at which frequencies these sounds are. The dB value is a simple calculation with 1 result. To calculate the frequencies we need a more complicated calculation and a much greater packet size. The communication will be done over LoRa which is a long range communication protocol. Its drawback is that it can only send few and small packets. This means it will be difficult to send the entire DFT. The dB value however can still be transmitted using LoRa.

To be able to capture the sound we will need to design a PCB containing a microphone and any additionally required components. These components will depend on wether we will work with an analog or a digital microphone. Besides this we will need to find a development board which is LoRa enabled and can also communicate with the microphone.

1.3 STRUCTURE OF THE MANUSCRIPT

The thesis will start off with Chapter 2 which will go over some knowledge that will be required to understand the working of this project. The main topics are What is IoT, the theory of sound, SPI communication, LoRa and DFT.

After this we will move on to Chapter 3. In this chapter we will go over the different components we choose and why. The chosen components will also be compared to some alternatives. Once we know what components will be used we will see how they fit together on a pcb.

Following the hardware chapter there will also be a software Chapter 4 which will go over the code. we will see what micropython is and the file structure it follows. After that we will go over the different python files. These files contain several steps, from creating the spi communication, configuring the ADC over SPI to actually capturing the data and processing it.

Finally we will discuss the final design and how it can be improved.

2.1 INTERNET OF THINGS

Internet of Things or IoT describes a network containing several low power sensors and 1 or more gateways. These gateways allow the sensors to send their data to the cloud from where it can be read from anywhere. Through IoT networks all nodes, even the really low power ones, gain access to the internet. Those networks allow us to have smart homes where we can change the thermostat with a phone app or activate the coffee machine while still in bed. IoT is not only big in home use but also in industrial use. It can be used in security to check if there are any open doors or windows, or something completely different like in agriculture to check the humidity and temperature of the ground.

IoT really is the future but it still has some problems to overcome. One of the main problems is the power consumption. Some sensors are directly connected to a powerline but in some cases that is not possible. In case of wireless sensors we rely on a batteries to keep the sensors powered. This means that we need to make sure the nodes don't consume more power then strictly required. There are some companies who create battery-free IoT nodes which gather their energy from the environment but this is not always possible. For this project we will need to use batteries which is why we will try to be energy conscience when choosing our components and writing the code.

2.2 SOUND

2.2.1 CHARACTERISTICS OF SOUND

In order to design a pcb to accurately measure sound we first need to understand what sound is and what characteristics we need to look out for. If the information below is insufficient a more detailed explanation can be found in [EXPLAINED, 2016](#).

sound is a vibration that propagates acoustic waves through a medium. These vibration can happen at different frequencies. The human ear can perceive waves with frequencies between 20 Hz and 20 kHz. As can be seen in [fig. 2.1](#) sound can also have frequencies under 20 Hz, infrasound, or above 20 kHz, ultrasound, but for this project we will focus on the human audible sound range.

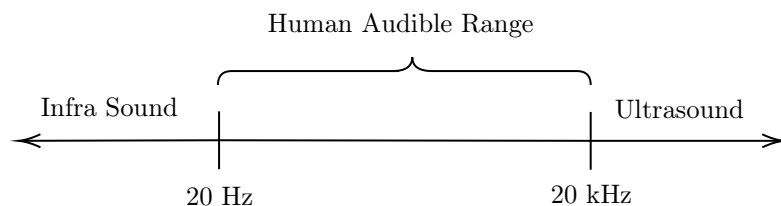


Figure 2.1: sound spectrum

2.2.1.1 Sensitivity

The sensitivity of a microphone is a measurement of the electrical response to a standard tone of 1 kHz at 94 dB SPL which equals a SPL of 1 Pa. The units of sensitivity depend on whether you are using an analog microphone or a digital microphone.

For an analog microphone sensitivity is expressed in dBV which uses a reference of 1 V rms. Since dB is a relative value we don't directly measure it, instead we measure the sensitivity in mV/Pa and convert it to dBV with formula 2.1.

$$\text{Sensitivity}_{\text{dBV}} = 20 \cdot \log_{10} \left(\frac{\text{Sensitivity}_{\text{mV/Pa}}}{\text{Output}_{\text{REF}}} \right) \quad (2.1)$$

where $\text{Output}_{\text{ref}}$ equals 1 V/Pa reference output ratio.

A digital microphone will express sensitivity in dBFS which uses 1 full-scale digital output as reference. The full-scale digital output is defined as the highest signal level that can be output by the microphone. For a digital microphone you can use formula 2.2.

$$\text{Sensitivity}_{\text{dBFS}} = 20 \cdot \log_{10} \left(\frac{\text{Sensitivity}_{\%FS}}{\text{Output}_{\text{REF}}} \right) \quad (2.2)$$

Visually the sensitivity can be seen as the difference between the 94 dB point and the noise floor.

It is also important to note that the chosen sensitivity depends on the task. A low sensitivity microphone will be able to pick up loud noises without too much ambient sound but will struggle with quieter noises. If you try to amplify this low noise signal you will get a very low quality signal.

On the other hand, high sensitivity microphones can be used when trying to capture all sounds. However, these microphones will struggle with sounds that get too loud.

2.2.1.2 Noise

Noise refers to unwanted modifications of the signal during capturing, processing and transmission. There are several types of noise with some being easier to deal with than others. A source of noise can be anything from a nearby device to the temperature.

2.2.1.3 Signal-To-Noise Ratio

As the name suggests SNR is a ratio between the signal power and the noise power and is expressed in dBA which means the value is A-weighted. This is a way to account for relative loudness due to the human ear not perceiving every frequency with the same sensitivity. A higher SNR means there will be more useful information than noise. If you for example have a SNR of 100 dBA it means that your signal will be 100 dBA higher than the noise.

2.2.1.4 Acoustic Overload Point

The AOP is the maximum value to which the microphone will respond linearly.

2.2.1.5 Equivalent Input Noise

The Equivalent Input Noise is the signal you would receive at the output of the mic if the noise would be seen as the input. It is used to specify noise levels.

2.2.1.6 Dynamic Range

The dynamic range of a microphone refers to the difference between the loudest and quietest SPL to which the microphone will respond linearly. When connecting an analog microphone to an ADC it is important to be sure the ADC's dynamic range is equal or larger than the microphone's. Otherwise you won't be able to fully utilize the microphone to the fullest extent.

To calculate the dynamic range you can simply subtract the EIN from the AOP.

We can look at fig. 2.2 to see how all of these characteristics fit together.

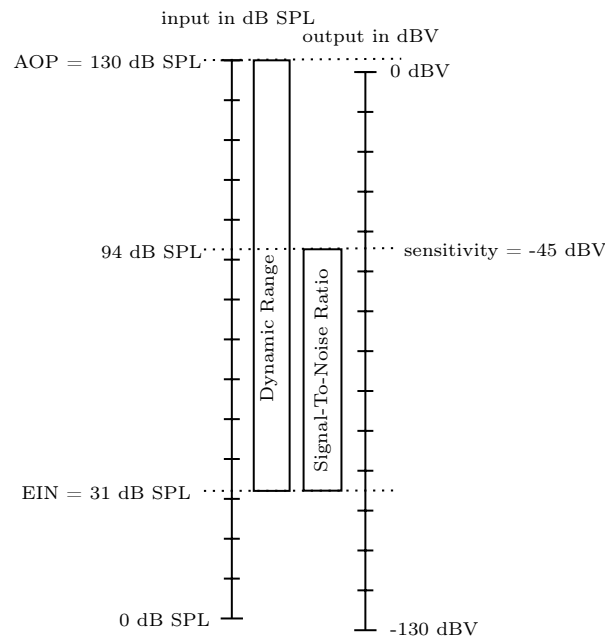


Figure 2.2: Relation between dB SPL input and dBV output for the ICS-40300

2.2.1.7 Frequency Response

The frequency response of a microphone will determine the output for a certain input frequency. The bottom and upper limit are the points at which the frequency response is -3 dB and is known as the -3 dB point. These limits are what we use to describe the frequency range.

After these -3 dB points there is a frequency roll-off which determines how sharp the cut off is. It is expressed in dB/decade or dB/octave. An ideal filter will have a roll-off of ∞ dB/decade.

2.2.1.8 Power Supply Rejection

Power Supply Rejection or PSR describes how much noise will be added to the signal from the power supply.

To determine this value a 217 Hz square wave with 100 mV peak-to-peak is applied to the Vdd pin of the microphone without an input signal. The output of the microphone is then A-weighted and integrated over a 20 kHz bandwidth. This will result in a negative number in dBV for analog microphones and dBFS for digital microphones.

If the microphone has complete PSR then the resulting value will be equal to the noise floor.

2.2.1.9 Directionality

Directionality describes the response depending on the direction of the sound wave. There are 2 options, unidirectional and omnidirectional.

Unidirectional microphones will have a normal response in 1 direction and close to no response from the other sides as can be seen in Figure 2.3. This microphone is made to measure sound coming from a specific source without capturing ambient noise.

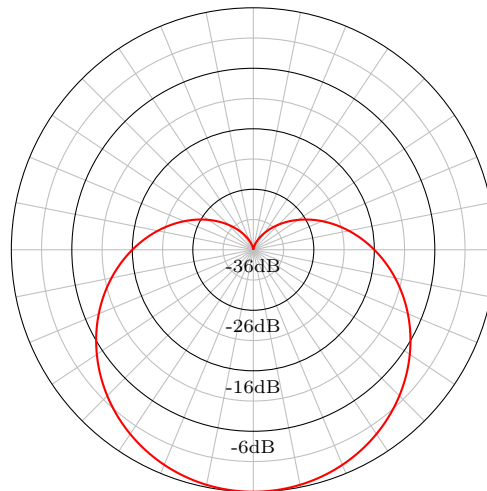


Figure 2.3: Frequency response unidirectional microphone

Omnidirectional microphones will have the same response no matter where the sound is coming from as can be seen in fig. 2.4. This microphone is used if you are more interested in the ambient sound.

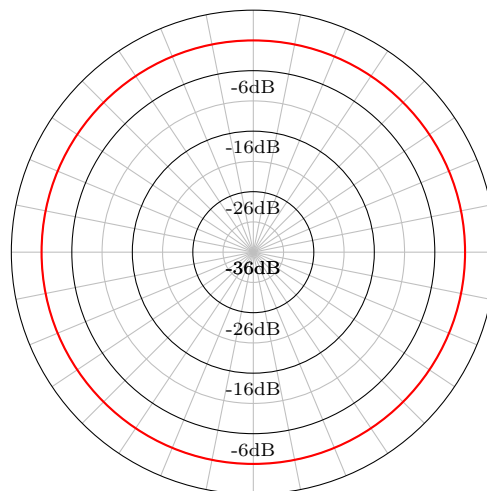


Figure 2.4: Frequency response omnidirectional microphone

2.2.2 MEMS vs ECM MICROPHONE

While it is important to understand the characteristic of sound we should also take a look at the microphones we use to measure it. There are 2 types of microphones we can use, an Electret Condenser Microphone(ECM) or a Micro-electro-mechanical Systems(MEMS) microphone.

The ECM is built out of an electret diaphragm, which is a material with a fixed surface charge, spaced close to a conductive plate in which a capacitor is formed with the air gap as the dielectric. Voltage across this capacitor varies according to the sound pressure waves moving the electret diaphragm. This voltage change is then amplified by an internal JFET. The entire build of the ECM can be seen in fig. 2.5.

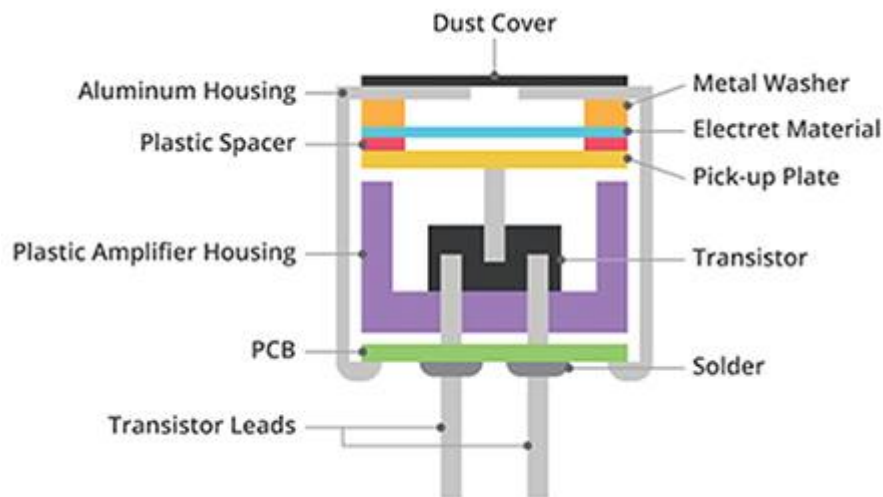


Figure 2.5: ECM built, source: [MEMS versus ECM 2023](#)

The other option we have is a MEMS microphone. This type of microphone is built with a MEMS component. It also has a small hole to allow the sound into the microphone. This hole can be through the cover, top-ported, or through the pcb, bottom-ported. The MEMS component is built with a mechanical diaphragm. Just like the ECM, the MEMS diaphragm forms a capacitor and sound pressure waves cause the change in signal. fig. 2.6 shows an example of a top-ported MEMS microphone.

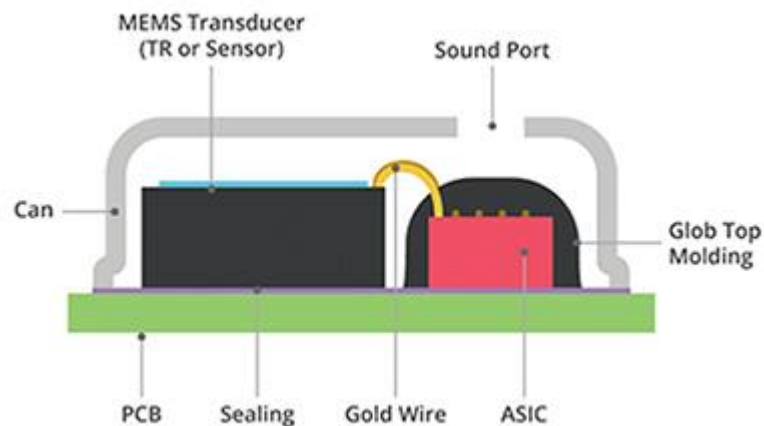


Figure 2.6: MEMS built, source: [MEMS versus ECM 2023](#)

A more in depth comparison can be found in [MEMS versus ECM 2023](#).

2.2.3 SAMPLING

Probably the most important concept when digitalizing analog signals is sampling frequency. when converting an analog continuous signal to a discrete digital signal we take multiple samples per second. The amount of samples taken is the sampling frequency or sampling rate and is measured in Hz.

Ideally your computer would record the entire analog signal but this goes against the concept of digitalization. The analog signal is continuous or in other words, has an infinite amount of samples. This is not possible to store in a computer.

The reason we need to sample at a high enough frequency is to prevent aliasing. This is a problem that arises when trying to reconstruct the original signal from samples. Aliasing is when we cannot differentiate between different multiples of the same frequency. As can be seen in Figure 2.7, if our samples are the purple dots, we cannot tell if the original signal was the blue line or the green line.

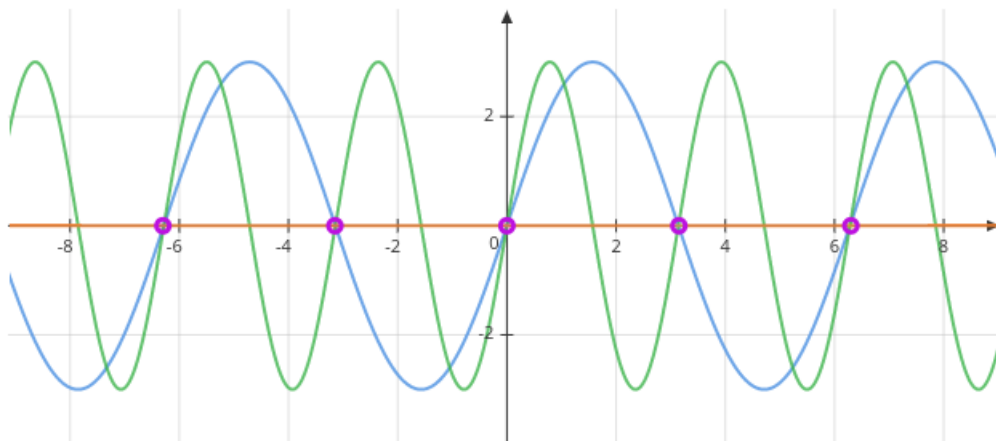


Figure 2.7: Aliasing example

However, if our sampling rate is high enough we are able to differentiate the 2 signals as can be seen in Figure 2.8.

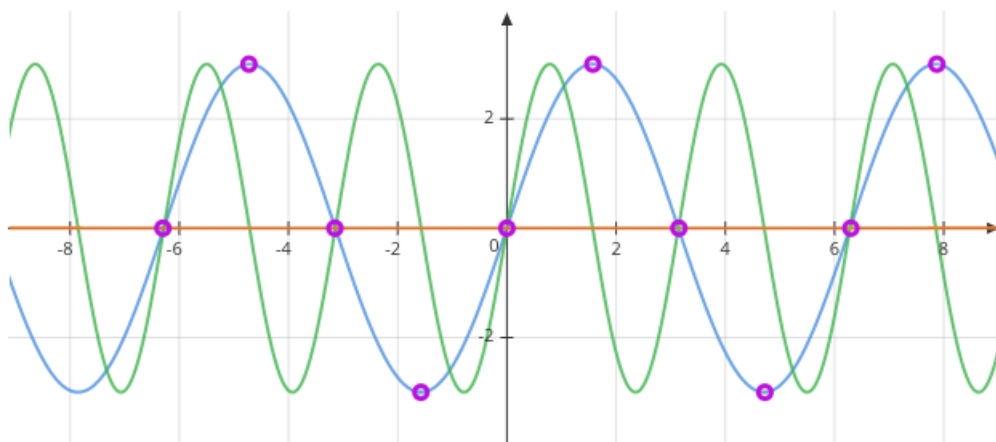


Figure 2.8: Aliasing example with higher sampling rate

Now the question is, what sampling rate is high enough? According to the Nyquist-Shannon sampling theorem, we need to sample at least twice the highest frequency in the signal. This is called the Nyquist frequency. Of course this also means we need to be sure of the highest frequency of the signal. If we for example have a microphone that measure from 20 Hz to 20 kHz we would need a sampling frequency of at least 40 kHz. However, our signal might also contain higher frequency noise which will influence the actual sampling frequency we need. Therefore, we add an anti-aliasing filter in front of the ADC. The ADC is what we use to convert the continuous analog signal to a discrete digital signal. This is a low pass filter with a cutoff frequency equal to the highest allowed frequency. This filter will prevent any higher frequencies from interfering with the signal.

Another way of looking at this problem is in the frequency domain. In Figure 2.9 you can see the frequency spectrum of a continuous analog signal S_c .

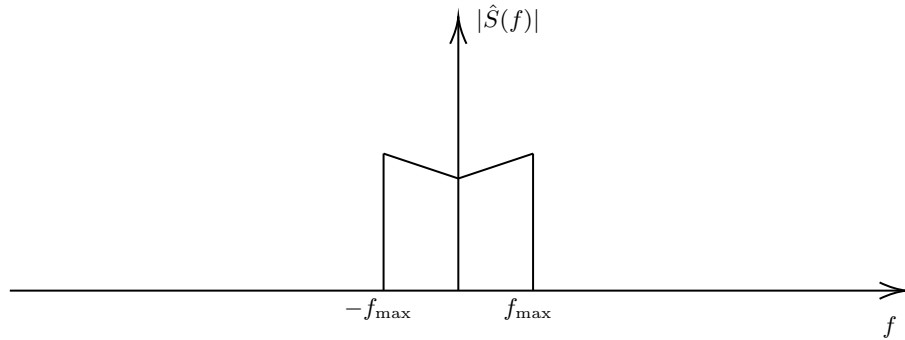


Figure 2.9: Frequency spectrum of S_c

If we then sample this signal we see something special happen. Figure 2.10 shows us the discrete frequency spectrum of the sampled signal S_d .

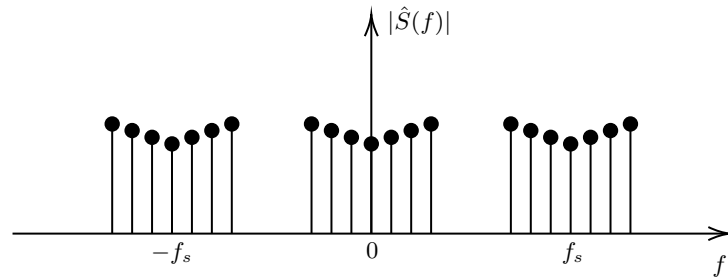


Figure 2.10: Frequency spectrum of discrete signal S_d

Instead of having a continuous frequency spectrum we have several discrete samples. These samples still form the same shape as the original spectrum. The big difference is that the spectrum is repeated around f_s , $-f_s$ and any other integer multiple of f_s . This phenomenon is called aliasing. To solve this problem we can simply place an anti-aliasing filter around the original signal and cut off the rest of the spectrum. However, when we don't choose a high enough sampling frequency we will not be able to differentiate between the original signal and the first alias. This is why we need to sample at at least the Nyquist frequency. An example of a sub Nyquist sampled signal can be seen in Figure 2.11.

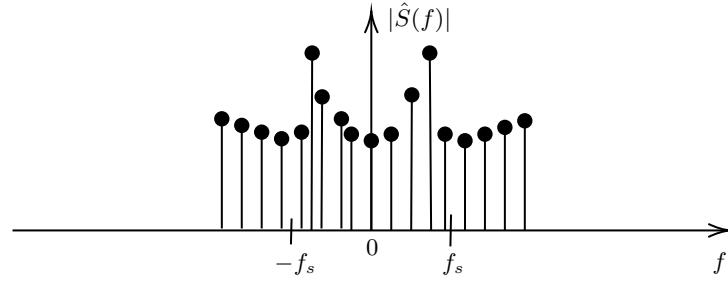


Figure 2.11: Frequency spectrum of discrete signal sampled at sub Nyquist frequency

As you can see, the spectral replicas are overlapping. There is not a single technique that can separate them once sampled at a sub Nyquist frequency. It is important to note that to keep the image clear we only show the replicas around f_s and $-f_s$. Not around every multiple of f_s .

If you are interested in a more interactive explanation of the Nyquist theorem and aliasing, you can check out Wilczek, [2019](#).

2.3 SPI PROTOCOL

This project will utilize the SPI communication protocol between the ADC and the development board. It is therefore important to understand how SPI works.

Serial peripheral interface or SPI is one of the most used interfaces for communication between microcontrollers and IC's such as sensors, ADC's, DAC's and many more. It is a synchronous full duplex interface which can be configured with 3 or 4 wires. This project will utilize the 4-wires configuration which can be seen in fig. [2.12](#).

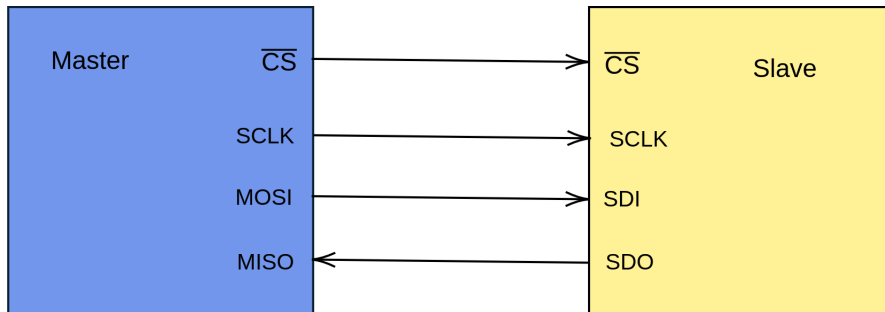


Figure 2.12: SPI communication between 2 nodes

In the simple case of trying to connect 2 nodes we will have 1 master node and 1 slave node. The 4 pins of the master will be:

- chip select($\overline{CS}$): the master will have 1 unique $\overline{CS}$ pin for every connected slave device.
- serial clock(SCLK): Every device will be connected to the same SCLK line for synchronization. The frequency of the clock is set by the master.
- Master Out Slave In(MOSI): The master will write its data to this line for the slave node to read.
- Master In Slave Out(MISO): The slave node will write its data to this line for the master to read.

For the slave there is a small difference. There is still a $\overline{CS}$ pin and a SCLK pin but the MOSI and MISO pins are replaced by SDI and SDO pins. SDI stands for Slave Device In and SDO stands for Slave Device Out. As the names would suggest you connect the MOSI pin of the master to the SDI of the slave and the MISO pin of the master to the SDO pin of the slave.

SPI can be run in 4 modes depending on the clock polarity(CPOL) and clock phase(CPHA) which can be set by the master. In order to work with certain components you need to make sure both devices support the same modes. CPOL determines the state of the clock while idle, this can be either logical low or logical high. CPHA determines whether to use the rising or falling edge for sampling and/or shifting the data.

Communications will always be initiated by the master. To start communication we need a $\overline{CS}$ falling edge, to end communication we need a $\overline{CS}$ rising edge. Or vice versa if you change the CPHA.

SPI can be configured to run with multiple slaves. This can be done 2 different ways. Either every device is connected to the SCLK, MOSI and MISO of the master and a unique $\overline{CS}$ pin, which can be seen in fig. 2.14, or the slaves are daisy-chained together as in fig. 2.13. While the first method will be faster it will also require more connections and pins. This will grow exponentially with the amount of nodes. Daisy-chaining will slow down the communication but will allow for a larger number of nodes.

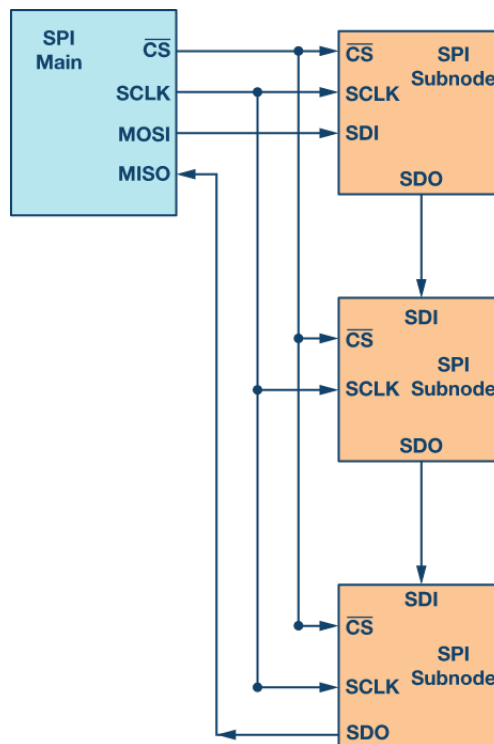


Figure 2.13: SPI daisy-chain setup

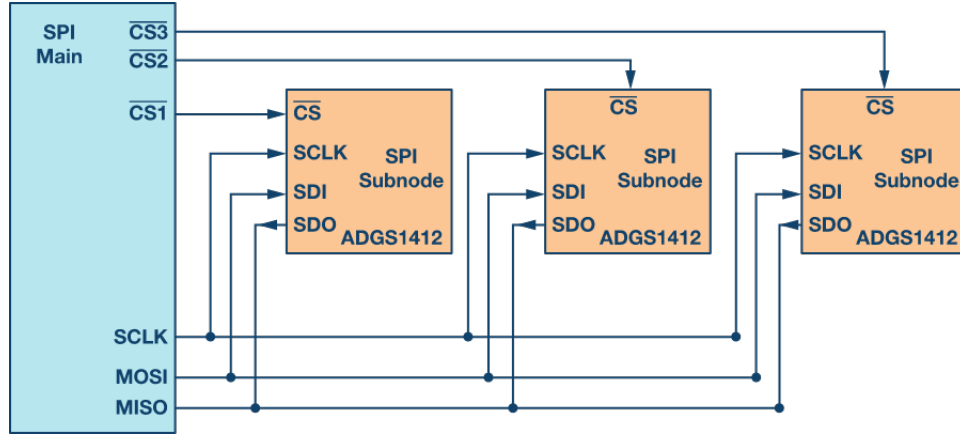


Figure 2.14: SPI multinode setup

2.4 LoRa AND LoRaWAN

LoRa is a modulation technique which makes use of chirp spread spectrum. This technique sweeps between a lower and an upper frequency over a certain time period. Increasing the bandwidth will allow you to send more data, however the amount is limited by law. In Europe you can use 125 kHz or 250 kHz. The other factor that can be changed is the spreading factor. A higher spreading factor will increase the resistance to interference and distance over which it can send at the cost of data rate. The spreading factor can be set between 7 and 12. Varying these 2 parameters will allow for a data rate between 250 bits/s and 11 kbits/s. Another big limitation to keep in mind is that you can only send for 1% of the time. This means that you can only send for 36 s per hour. It is important to note that these numbers are for Europe where the resources of LoRa are more limited by law.

LoRaWAN is a MAC layer protocol which makes use of LoRa as a modulation technique. This allows devices to build and maintain a network using LoRa. It is developed by the LoRa Alliance, a non-profit association.

A more detailed explanation can be found on the things network website at [What Are LoRa and LoRaWAN? 2023](#).

2.5 DFT

The Discrete Fourier Transform or DFT is one of the most important transforms in signal processing. To explain it we first need to understand the Fourier Transform or FT. The FT is a mathematical transform that decomposes a signal into a weighted sum of sinusoids as can be seen in Figure 2.15.

This allows us to analyze the frequency content of a signal. A big problem with the FT is that it requires a continuous signal going from $-\infty$ till $+\infty$. This is not possible in the real world. In our case we will be working with sound measurements. Our signal will be a discrete signal of 512 samples. In order to calculate the FT of this we need to use the DFT. This is a version of the FT that can be applied to discrete signals. The way it works is by looking at our samples as 1 period of an infinite signal. We can then write the signal as a modulo function where our samples are repeated. Modulo will allow us to use some properties like shift right and circular shift. There is far more that goes into the DFT but this is the basic idea. A far more detailed explanation can be found in Chapter 8 of the book by Smith, 1999. The formula we will need

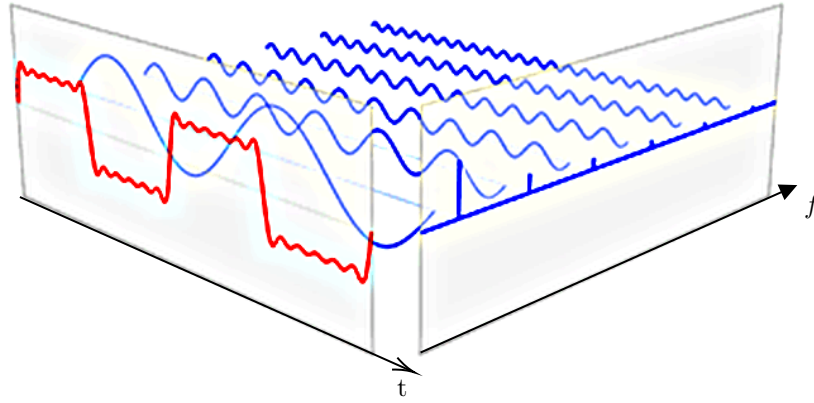


Figure 2.15: Fourier Decomposition, adepthed from Kordahl, [2021](#)

to calculate the DFT is

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}. \quad (2.3)$$

In terms of hardware this project can be split into 2 parts:

- a LoRa enabled development board
- a PCB which is capable of capturing sound

3.1 DEVELOPMENT BOARD

The development board is the component with the most restrictions. This is because there are certain restrictions implied by the goal of this project.

- LoRa communication capabilities
- low-power performance
- processing power to enable DFT calculations

With these restrictions in mind we looked for acceptable boards. Our promoter pointed us towards pycom which is a company that has both hardware and a cloud service for IoT. With this extra restriction there were 2 possible choices: the LoPy4 or the Fipy. When comparing them in table 3.1 we can see that these boards are almost identical. The only difference is that the FiPy offers more types of communication. This also makes the FiPy more expensive. Since the LoPy4 is sufficient for this project there is no reason to go with the FiPy.

Table 3.1: Comparison development boards: LoPy4 vs FiPy

	LoPy4	FiPy
Price	40 €	80 €
LoRa current active	± 100 mA	± 100 mA
LoRa current sleep	1 μ A	1 μ A
clock frequency	160 MHz	160 MHz
flash memory	8 MB	8 MB
RAM	512 KB + 4 MB	512 KB + 4 MB

Unfortunately due to change in ownership at pycom, production of all boards have been temporarily halted. This means it was difficult to find their products in stock. The LoPy4 was nowhere to be found but with the fipy being a more expensive board it was still available at certain vendors. This is why the project will be built with a fipy. This project can be recreated with a LoPy4 and achieve the same results.

To not get locked ourselves to pycom we decided to look for similar boards from other vendors. A board that is used in a lot of LoRa projects is the raspberry pi pico. You can find a comparison with the LoPy4 below in table 3.2.

Table 3.2: Comparison development boards: LoPy4 vs Raspberry Pi Pico

	LoPy4	Raspberry pi pico
Price	40 €	5 €
LoRa current active	$\pm 100\text{mA}$	$\pm 100\text{ mA}$
LoRa current sleep	1 μA	1.3 mA
clock frequency	160 MHz	133 MHz
flash memory	8 MB	2 MB
RAM	512 KB + 4 MB	256 KB

Overall the LoPy4 is a better board. Of course the price reflects this fact. The raspberry pi pico is still a very nice board for its price but the lower processing power and higher current draw while idle are enough reasons to justify paying more for the LoPy4. The reason for this is that the nodes will spend most of their time sleeping while only waking up once every 15 minutes to measure the sound. This means the raspberry pi pico's higher current draw while in sleep mode will consume a lot more power than the LoPy4.

3.2 COMPONENTS

The pcb will need to be able to measure sound accurately and send it to the development board. The 2 main components it needs to achieve this goal are a microphone and an ADC in case of an analog microphone.

3.2.1 MICROPHONE

For the microphone we have chosen the '[ICS-40300](#)' [2023](#). Lets have a look at its characteristics and why we choose them.

- The first choice we have to make is whether we will use an analog or a digital microphone. We decided to go with an analog microphone since it is cheaper and will allow us to learn more about designing a pcb than a digital microphone can.
- The second big decision is whether we go with a MEMs microphone or an ECM. MEMs is the newer and better technology, the only reason to still use ECM's are previous dependencies which we don't have. We will use a MEMs microphone.
- The next thing we look at is the directionality of the microphone. as stated in section [2.2.1.9](#) we have 2 choices. Since there is no specific source we want to measure, but instead we the sound coming from all directions we will go with an omnidirectional microphone.
- Another important specification to choose is the frequency range. Since we want to match the human hearing we will want a microphone that can measure between 20 Hz and 20 kHz. The ICS-40300 has a frequency range of 6 Hz - 20 kHz.
- We will now look at some important characteristics like the microphones sensitivity. To choose this value we need to look at what we want to measure. We care about sound pollution so we care most about medium to loud noises. However, we still want to be able to have some idea of the quieter sounds. This is why we need a medium sensitivity microphone. The theory behind this is explained in section [2.2.1.1](#). The ICS-40300 has a sensitivity of $-45\text{ dBV} \pm 2\text{ dBV}$.
- We will be working with a big range of sounds, which means it is even more important to have a low SNR. This is because we will need to amplify the signal but don't want the noise to get out of hand. The ICS-40300 has a SNR of 63 dBA.

- The next choice we have to make is the range of our microphone. As said previously we are most interested in medium to loud sounds. But how do we define this? As stated in [What Noises Cause Hearing Loss? 2022](#) sound can cause instant pain and damage around the 120dB. That means this is the loudest sound we care about since we are most interested in noise pollution causing damage over long term, not short burst of sound which cause instant damage. To have some margin of error, we will look for a mems microphone with an AOP a little higher than 1200 dB SPL

We then have to choose a lower boundary. To choose this value we should look at what level normal background sounds happen. According to [What Noises Cause Hearing Loss? 2022](#), a refrigerator hums at around 40 dB. This is an acceptable sound level to measure for sound pollution. To find the lower limit of a microphone we subtract the DNR from the AOP.

The ICS-40300 has a AOP of 130 dB, a DNR of 99 dB and thus a lower limit, or EIN, of 31 dB.

- PSR is always important when working with signals. However, MEMs microphones always have a good PSR so we don't really need to worry about the exact value. The ICS-40300 has a PSR of -80 dBV.

3.2.2 ADC

To fully utilize the dynamic range of the microphone the ADC has to match the dynamic range. The dynamic range of the ADC can be calculated with

$$\text{DNR} = 6.021 \cdot N + 1.763 \text{ dB} \quad (3.1)$$

where N equals the number of bits in the ADC. The onboard ADC for the LoPy4 has a 12 bit ADC. This gives us a dynamic range of 74 dB which does not match the microphone with a DNR of 99 dB. If we add an external 16 bit ADC we will have a DNR of 98 dB which is almost identical to the microphone. This 1 dB loss is acceptable since a 20 bit ADC will bring the current price of 3 € for a ADC to a minimum of 11 €.

You can convert the signal with a simple analog ADC but for this project we decided to go with the 'MCP3461' [2020](#) which is a more complex digital ADC with the following capabilities:

- 2 channels
- Programmable data rate up to 153.6 ksp/s
- Programmable gain ranging between 0.33X and 64X
- SPI communication
- continuous conversion mode which can be turned off to save power

Due to the programmable gain of the adc we will not need an opamp and thus the pcb will be smaller. We will however have a more complex code since we will have to program the configuration of the ADC.

3.2.3 LOW PASS FILTER

As previously mentioned in Section 2.2.3, we need a LPF to ensure we don't have aliasing. With the microphone measuring up to 20 kHz, we need a LPF with a cutoff frequency of 20 kHz. In the datasheet of the MCP3461 it is also recommended to add a LPF before every analog input

channel in order to remove the noise from the digital components. However, when originally designing the PCB we added a first order RC filter. This turned out to be a bad decision. A first order RC filter has a roll off of -20 dB/decade. While this is not sharp at all, it isn't a big issue since we want to filter out the frequency noise generated by the digital components which is in the MHz. The main issue with this filter was the noise it created itself. When originally testing we saw a very large noise floor which did not match our expectations. After looking further into it, we found out that a RC filter has Johnson-Nyquist noise. This noise turned out to be so large we could not longer accurately measure anything besides very loud sounds. We then tried the PCB without the LPF and found far better results. While a LPF is really important, we continued the project without one. More research is required to find a low noise LPF which is simple to implement on a small PCB.

3.3 PCB

In Figure 3.1, Figure 3.2 and Figure 3.3 you can see the schematics of the pcb. As you can see their are more components then just the microphone and the ADC. Lets take a look at those extra components.

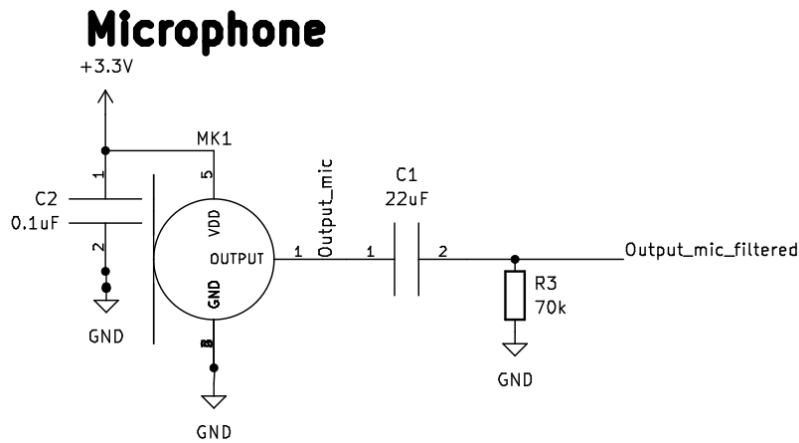


Figure 3.1: schematic of the microphone

Both the microphone and the adc have decoupling capacitors at their power pins. For the microphone the datasheet recommends a simple 0.1 μF capacitor. For the ADC we use a 10 μF capacitor in parallel with a 1 μF capacitor.

The microphone in Figure 3.1 also has a capacitor at the output pin to remove the DC component. We do this because we are only interested in the AC signal and keeping the DC component of the signal will limit how much we can amplify the signal.

After this capacitor we see a resistor to the ground. This resistor is required to allow the previously mentioned capacitor to charge. It is important to charge the capacitor otherwise a part of the DC component of the signal will get through. If there is no resistor the only time there is current running through the capacitor is when the ADC is actively capturing data. This is a problem because we do not want to wait 100+ cycles for the capacitor to charge before we get accurate readings.

Now let's take a look at Figure 3.2 to see what additional components the ADC requires. The first thing we see is a voltage divider with R1 and R2. This voltage divider is used to set REFIN^+ of the ADC. It can be calculated as follows:

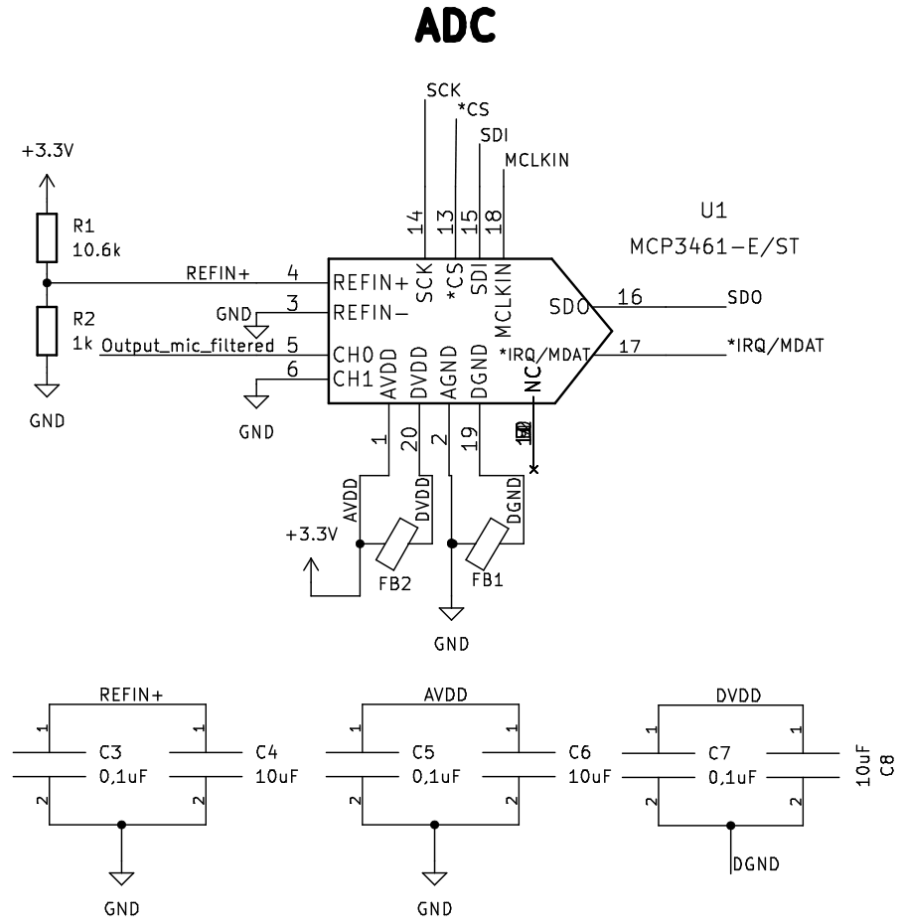


Figure 3.2: schematic of the ADC

First let's see what the highest possible output is of the microphone. According to the datasheet this value is 0.355 V rms.

If we choose a GAIN of x8 this results in a maximum output of 2.84 V. Ideally you would match this value to V_{cc} but in this case our GAIN is limited by the set of GAIN settings of the ADC. The closest we can get to 3.3 V without going above it is by using the x8 GAIN setting.

now we use the following formula to calculate the voltage divider:

$$V_d = \frac{V_{cc} \cdot R_2}{R_1 + R_2} \quad (3.2)$$

If we apply a V_{cc} of 3.3 V and choose $R_2 = 1 \text{ k}\Omega$ we find that $R_1 = 10.6 \text{ k}\Omega$ for $V_{des} = 2.84 \text{ V}$.

Just like the power pins, the REF+ pin also gets 2 decoupling capacitors to try and keep the signal as steady as possible.

The adc we choose works in both the analog and digital domain. This means it also has 2 separate power and ground pins for analog and digital. This is done to prevent the noise generated by digital components in the ground plane from reaching the analog components. However, to fully separate them it would require 2 separate power sources which is something we cannot do in an IoT device. This is why we use ferrite beads. A ferrite bead is a type of filter that suppresses high frequencies. These components allow us to use the same power source while still separating the analog and digital components. We place 1 ferrite bead between the analog V_{cc} , which is directly connected to 3.3 V, and the digital V_{cc} . Same goes for the ground

pins, the analog ground is connected directly to the power source ground and the digital ground is connected to the analog ground through a ferrite bead.

Connectors

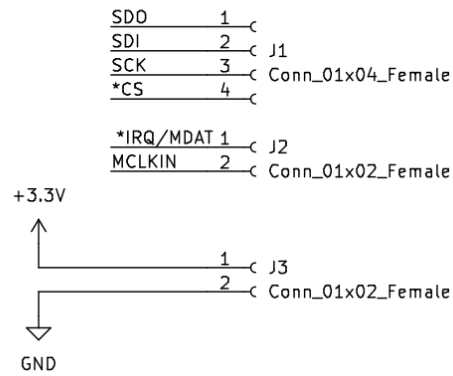


Figure 3.3: schematic of the connectors

Finally, to connect the pcb to our development board we add a couple of connectors. We need to connect 2 power pins, 4 spi pins and then we have 2 additional pins which give us more control over the ADC.

If we add all of these parts on 1 pcb it will look as follows:

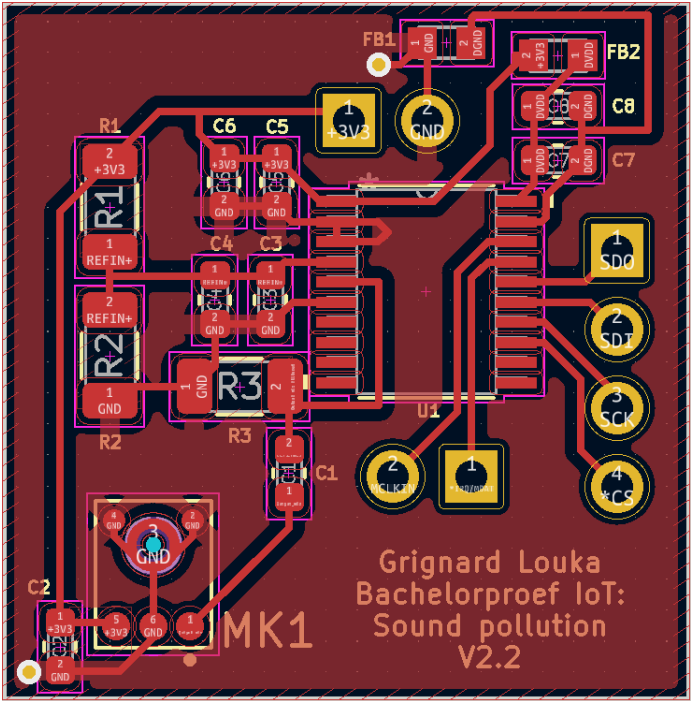


Figure 3.4: 2D front view

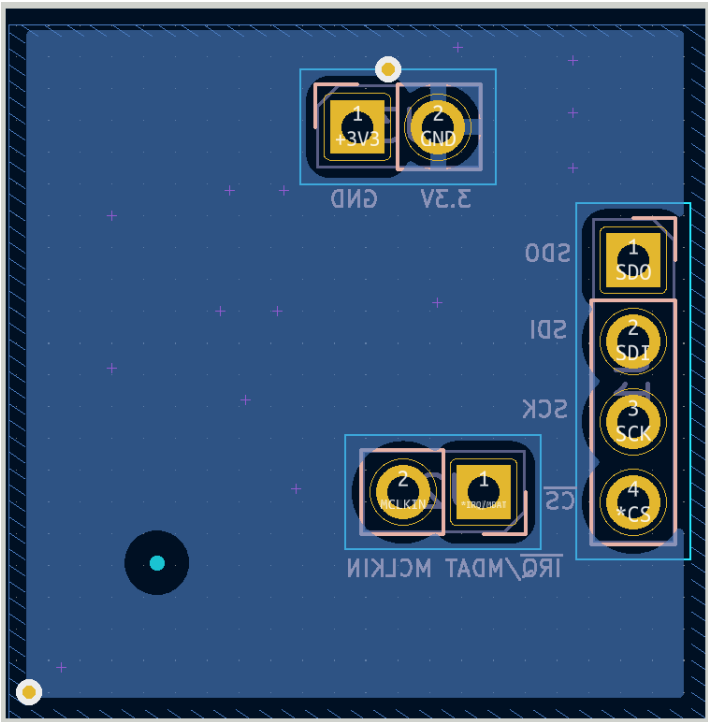


Figure 3.5: 2D back view

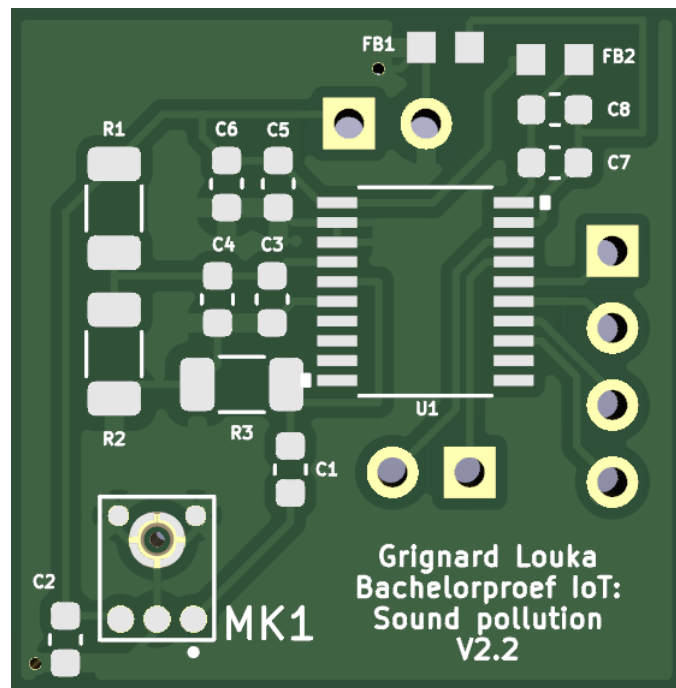


Figure 3.6: 3D front view

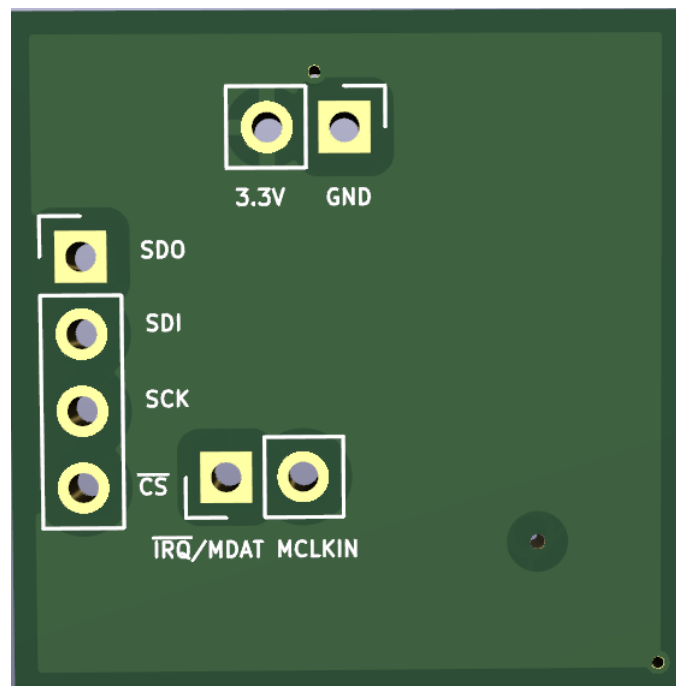


Figure 3.7: 3D back view

4.1 MICROPYTHON

One of the reasons we choose to work with a development board from pycom was the integration with micropython. Micropython is a lightweight, open source version of python that runs well on low power devices. All you need is a python file and the Pymakr extensions to upload the code and you're good to go.

The code gets uploaded to /flash on the development board. The 2 most important files are boot.py and main.py which are run automatically at boot. Besides these, you can choose the file structure. They do give you some empty folders to help. boot.py and main.py are 2 normal python files which can contain any code. If you want, you can run your entire software in boot.py. Ofcourse, this is not a good way to code which is why we will separate our code into the correct files.

4.2 CODE

4.2.1 BOOT.PY

Lets start off by taking a look at the boot file in Listing 4.1. This is the first file that will be ran on the development board.

Listing 4.1: Boot.py

```
1  # Message to indicate the start of the boot file
2  print("\n\n\nStarting up fipy")
3
4  # Turn off default flashing LED
5  pycom.heartbeat(False)
6
7  # Custom flashing LED
8  pycom.rgbled(0xFF0000) # LED Red
9  time.sleep(1)          # wait 1 second
10 pycom.rgbled(0x000000) # LED Off
11
12 # initialize spi communication and CS pin
13 spi, cs = spi_init()
14
15 # write the correct configuration to the ADC
16 write_config(spi, cs)
17
18 # Message to indicate the end of the boot file
19 print("BOOT COMPLETE")
20
21
```

Our boot file does several things:

- print message to indicate the start of the boot file

- turn of the default flashing LED
- custom flash the onboard LED to visually indicate the start of the boot file
- initialize the SPI communication
- write the correct configuration to the ADC over SPI
- print message to indicate the end of the boot file

The boot file calls two external functions, `spi_init` and `write_config`. These functions are defined in `ADC.py` which contains all the functions to interact with the ADC. Inside `spi_init` which can be seen at Listing 4.2 we initialize the $\overline{\text{CS}}$ pin and SPI on the development board with the correct pins and baudrate. We return the spi connection and the $\overline{\text{CS}}$ pin so they can be used.

Listing 4.2: Python function to initialize everything required for SPI communication

```

1
2  def spi_init():
3      # INIT PINS
4      cs = Pin(PIN_CS, mode=Pin.OUT, value=1)
5
6      # INIT SPI
7      spi = SPI(1)
8      spi.init(mode=SPI.MASTER, baudrate=9_830_400,
9              pins=(PIN_SCK, PIN_MOSI, PIN_MISO))
10
11     return spi, cs
12

```

After having initialized the spi communication, we use it to write the correct configuration to the ADC. This function can be seen in Listing 4.3. Before going over the function we first need to know how we communicate using SPI.

The first thing we do is send a command byte. This is always the first byte that is send after releasing $\overline{\text{CS}}$. The command byte follows the structure shown in Figure 4.1.

CMD[7]	CMD[6]	CMD[5]	CMD[4]	CMD[3]	CMD[2]	CMD[1]	CMD[0]
Device Address		Register Address / Fast Command bits				Command Type	
Bits						Bits	

Figure 4.1: command byte structure, source ‘MCP3461’ 2020.

The first 2 bits specify the device address. This is a hardcoded value which depends on the component. For the MCP3461 this value is 01.

The following 4 bits depend on the last 2 bits as can be seen in Figure 4.2. The last 2 bits specify the type of command. There are 4 types: fast command, static read, incremental write and incremental read. fast command is a way of configuring the ADC with only 1 byte. The exact command is specified by the 4 middle bits. All other types require additional bytes to be send. static read is used to continuously read 1 register specified by the 4 middle bits. incremental read/write automatically increase the register pointer after every operation and start at the specified register.

It is important to note that incremental write and incremental read don’t follow the same sequence of registers. While incremental read will go over all 16 registers, incremental write will not go over the first register and the last 2 registers. This is because you never have to write to these registers.

CMD[5:2]	CMD[1:0]	Command Description
0xxx	00	Don't Care
100x	00	Don't Care
1010	00	ADC Conversion Start/Restart Fast Command (Overwrites ADC_MODE[1:0] = 11)
1011	00	ADC Standby Mode Fast Command (Overwrites ADC_MODE[1:0] = 10)
1100	00	ADC Shutdown Mode Fast Command (Overwrites ADC_MODE[1:0] = 00)
1101	00	Full Shutdown Mode Fast Command (Overwrites CONFIG0[7:0] = 0x00)
1110	00	Device Full Reset Fast Command (Resets Whole Register Map to Default Value)
1111	00	Don't Care
ADDR	01	Static Read of Register Address, ADDR
ADDR	10	Incremental Write Starting at Register Address, ADDR
ADDR	11	Incremental Read Starting at Register Address, ADDR

Figure 4.2: command byte

Listing 4.3: write_config function

```

1
2     # REGISTER VALUES
3     CONFIG0 = 0xE2
4     CONFIG1 = 0x00
5     CONFIG2 = 0xA3
6     CONFIG3 = 0xC1
7     IRQ = 0x77
8
9     def write_config(spi, cs):
10         print("WRITING SETUP VALUES TO REGISTERS")
11         buff = bytearray(1) # buffer to store responses in
12         cs(0) # Start communication
13
14         # send command byte, store response in buff
15         spi.write_readinto(bytes([0x46]), buff)
16         print("Response command byte: ", hex(buff[0]))
17
18         spi.write(bytes([CONFIG0])) # CONFIG0
19         spi.write(bytes([CONFIG1])) # CONFIG1
20         spi.write(bytes([CONFIG2])) # CONFIG2
21         spi.write(bytes([CONFIG3])) # CONFIG3
22         spi.write(bytes([IRQ])) # IRQ
23         spi.write(bytes([0x08])) # MUX
24         spi.write(bytes([0x00])) # SCAN
25         spi.write(bytes([0x00]))
26         spi.write(bytes([0x00]))
27         spi.write(bytes([0x00])) # TIMER
28         spi.write(bytes([0x00]))
29         spi.write(bytes([0x00]))
30         spi.write(bytes([0x00])) # OFFSET CALIBRATION
31         spi.write(bytes([0x00]))
32         spi.write(bytes([0x00]))
33         spi.write(bytes([0x80])) # GAIN ERROR CALLIBRATION
34         spi.write(bytes([0x00]))
35         spi.write(bytes([0x00]))
36         spi.write(bytes([0x90])) # RESERVED
37         spi.write(bytes([0x00]))
38         spi.write(bytes([0x00]))
39         spi.write(bytes([0x50])) # RESERVED
40         # LOCK (Anything except A5 locks all register values)
41         spi.write(bytes([0xA5]))
42
43         cs(1) # End communication
44
45

```

Looking at the code in Listing 4.3 we can see that when writing the initial configuration we use incremental write mode, starting at register 0x01. The command byte for this action is 0x46. Every successful command byte will be responded to with a status byte. The status byte allows us to see certain flags without having to read the IRQ register. The status byte always starts off with two '0' bits to give other devices time to respond to avoid bus contention. The next three bits will confirm the hardcoded device address by repeating it and then adding the reversed last bit. The last three bits are flags for Data ready interrupt, CRC checksum error on the register map interrupt and POR interrupt.

After having send the command byte, and thus having set the ADC in incremental write mode starting at register 0x01, we start to send the values we want to place in the registers.

The datasheet contains a list of all possible configurations for every register. We will only go over the ones we use.

CONFIG0

The first value send is 0xE2 for register 0x01 which is also named CONFIG0. 0xE2 can also be written as 11 10 01 00. This will set 4 different settings.

- bit 7-6: if this value is 00, and all other bits in CONFIG0 are to, the ADC will go into shutdown mode. we set it to 11.
- bit 5-4: this determines the selected clock, 10 sets it to use the internal clock and to not show the clock on an output pin.
- bit 3-2: 00 is chosen to not add any current source to the ADC input pins.
- bit 1-0: 00 is chosen to keep the ADC in shutdown mode until we need to start capturing data.

CONFIG1

After writing 0xE2, the pointer will automatically go to the next register: CONFIG1. This time we send 0x00 or 00 0000 00.

- bits 7-6: 00 specifies a prescaler value for the clock of x1.
- bits 5-2: This sets the oversampling ratio. 0000 is equal to a OSR of 32.
- bits 1-0: these bits should always be set to 00

The OSR together with the Digital Master Clock or DMCLK determine the sample rate. The OSR is the value we select in CONFIG1 which in this case is 32. The DMCLK is dependent on the Analog Master Clock or AMCLK. The AMCLK is dependent on the Master Clock or MCLK. In this case we have selected the internal MCLK which is 4.9152 MHz. To calculate the AMCLK from the MCLK we simple divide the MCLK by a prescaler value which we can select in CONFIG1. In our case this value is 1 so the AMCLK is equal to the MCLK. Now to calculate the DMCLK we divide the AMCLK by 4. This gives us

$$\text{DMCLK} = \frac{\text{AMCLK}}{4} = \frac{\text{MCLK}}{4} = \frac{4.9152 \text{ MHz}}{4} = 1.2288 \text{ MHz}. \quad (4.1)$$

To determine the data rate we simply divide the DMCLK by the OSR. This gives us

$$\text{Data rate} = \frac{\text{DMCLK}}{\text{OSR}} = \frac{1.2288 \text{ MHz}}{32} = 38.4 \text{ kHz}. \quad (4.2)$$

As we have seen in Section 2.2.3 our sampling rate should be at least twice the highest frequency in the signal. with a maximum frequency of 20 kHz we would need a data rate of at least 40 kHz which we don't have in this case. Our data rate is close enough to 40 kHz that it won't cause a lot of problems but it is something to keep in mind. A solution is discussed in Chapter 5.

CONFIG2

The next byte send is 0xA3 or 10 100 0 11 to CONFIG2.

- bits 7-6: 10 sets the bias current to x1
- bits 5-3: 100 sets the GAIN to x8
- bit 2: This bit determines whether the ADC uses a auto-zeroing algorithm. We set it to 0 to disable it.
- bits 1-0: These bits should always be set to 11

CONFIG3

The next byte send is 0xC1 or 11 00 0 0 0 1 to CONFIG3.

- bits 7-6: 11 sets the ADC to continuous conversion mode.
- bits 5-4: These bits specify the ADC output format. 00 sets the format to 16 bits without allowing overrange.
- bit 3: this bit specifies the checksum format. Since we don't use checksums we don't care about this bit. We leave it at its default value of 0.
- bit 2: This bit enables checksum. Setting it to 0 turns it off.
- bit 1: this bit enables digital offset calibration. We leave it off by writing a 0.
- bit 0: this bit enables GAIN. We set it to 1.

IRQ

The next register is the interrupt register. We write 0x77 or 0111 0 1 1 1 to it. For the first 4 bits we don't care about the values and just leave them at their defaults. We do set the next 4 bits.

- bits 3: specify what data is put on the IRQ/MDAT pin. In this case we choose to output all interrupts.
- bit 2: setting this bit to 1 sets the inactivate state of the IRQ pin to logical high.
- bit 1: setting this bit to 1 enables fast commands.
- bit 0: setting this bit to 1 enables conversion start interrupt output.

MUX

The next byte we write is to the MUX register which determines what pins to use for Vin+ and Vin-. we write 0x08 or 0000 1000 to this register. This sets Vin+ at ch0 and Vin- at ch1. These are also the default values.

SCAN

After the MUX register we set the SCAN register. This register has to be set to 0x00 otherwise we will be running the ADC in scanning mode which is not what we want.

Other registers

After the MUX register we continue to write the default values to the remaining registers to make sure they are set correctly. These registers are TIMER, OFFSETCAL, GAINCAL, RESERVED and RESERVED.

LOCK

Finally when we have have written the correct configuration we lock the registers by changing the value in the LOCK register to something else then 0xA5 and then we pull $\overline{CS}$ high to end the spi communication.

This is the end of the boot.py file and also the beginning of the main.py file.

4.2.2 MAIN.PY

Inside the main.py file we have a while true loop which runs every 15 minutes. Inside the loop we do the following:

- set the ADC mode to conversion using a fast command
- read the ADCDATA register at address 0x00 a total of 512 times. The data is stored in an array.
- set the ADC mode to shutdown using a fast command
- Calculate the average loudness in dB SPL
- Calculate the FFT of the data
- Turn on a green LED for 1 second to indicate a successful loop.
- put the device in sleep mode for 15 minutes repeat back from step 1.

Listing 4.4: Main.py file

```

1
2     if __name__ == "__main__":
3         while True:
4             FC_conversion(spi, cs) # Put ADC in conversion mode
5             measurements = read_ADCDATA(spi, cs) # Read 512 measurements
6             FC_shutdown(spi, cs) # Put ADC in shutdown mode
7
8             dB = average_dB(measurements) # Calculate the average dB value
9
10            y = dft_ulab(measurements) # Calculate DFT with ulab library
11
12            # Transmit dB and y over LoRa
13            # Insert LoRa code here
14
15            # Flash LED to indicate a successful routine
16            pycom.rgbled(0x00FF00) # Green # DEBUGGING PURPOSES
17            time.sleep(1) # DEBUGGING PURPOSES
18            pycom.rgbled(0x000000) # Off # DEBUGGING PURPOSES
19
20            machine.sleep(15 * 60 * 1000)
21

```

4.3 DATA PROCESSING

The captured data is used to calculate 2 things, the loudness in dB SPL and the frequency spectrum. In the following sections we will explain how we calculate these values.

4.3.1 CALCULATING THE SPL IN dB

Since the goal of this project is to capture data about sound pollution it is important to correctly calculate the loudness of the sound. To convert the measured analog value to dB SPL we use the formula

$$\text{output} = 20 \cdot \log_{10} \left(\frac{V_m}{V_{\text{ref}}} \right) \quad (4.3)$$

where V_m is the measured voltage and V_{ref} is the reference voltage.

When working with an analog microphone the V_{ref} is given for a 1 kHz signal of 94 dB SPL. We also need to account for the GAIN we add to the signal. This transforms the formula into

$$\text{output} = 94 \text{ dB} + 20 \cdot \log_{10} \left(\frac{V_m}{V_{\text{ref}}} \right) - \text{GAIN} \quad (4.4)$$

where the GAIN is the total gain of the system in dB. To use this formula we need to calculate V_m , V_{ref} and the GAIN in dB.

To calculate V_m we convert the integer value from the ADC to a voltage. This is done by using the formula

$$V_m = (\text{measurement} / \text{ADC}_{\text{res}}) \cdot V_{\text{ref}} \quad (4.5)$$

where ADC_{res} is the resolution of the ADC which is the highest possible integer value it can output. In our case we have a 16 bits ADC which measure 65536 different values between -32768 and 32767 . V_{ref} is the reference voltage we choose which is 2.84 V. The measurement is the integer value we get from the ADC.

This gives us

$$V_m = (\text{measurement} / 32768) \cdot 2.84 \quad (4.6)$$

To calculate V_{ref} we need to know the sensitivity of the microphone. This is the voltage the microphone outputs when it measures 94 dB SPL. In [‘ICS-40300’ 2023](#) we find that the ICS-40300 has a sensitivity of -45 dBV at 1 kHz, 94 dB SPL. To convert this value to V we take the reverse logarithm. This gives us

$$10^{-45 \text{ dB V} / 20} = 0.005623 \text{ V}. \quad (4.7)$$

In this case 0.005623 V tells us what the output of the microphone will be for a 1 kHz signal of 94 dB SPL. If the microphone outputs double this value, 0.011246 V, we know that the signal is increased by 6 dB and is thus equal to 100 dB SPL. If the microphone outputs half this value, 0.002812 V, we know that the signal is decreased by 6 dB and thus 88 dB SPL.

Finally we need to convert the GAIN to dB with formula [4.8](#).

$$\text{GAIN} = 20 \cdot \log_{10} \left(\frac{V_{\text{out}}}{V_{\text{in}}} \right) \quad (4.8)$$

In our case we have a GAIN of 8 which gives us

$$\text{GAIN} = 20 \cdot \log_{10} (8) = 18.06 \text{ dB} \quad (4.9)$$

When we combine V_m , V_{ref} and the GAIN in dB into formula 4.4 we get

$$\text{output} = 94 \text{ dB} + 20 \cdot \log_{10} \left(\frac{\frac{\text{measurement}}{32768} \cdot 2.84}{0.005623 \text{ V}} \right) - 18.06 \text{ dB.} \quad (4.10)$$

4.3.2 CALCULATING THE DFT

Besides knowing the loudness of the sound it is also important to know what frequencies these sounds are at. To know this we have to calculate the Discrete Fourier Transform or DFT. The DFT is not a simple calculation, especially in python it is going to be slow. One way of speeding the calculation up is by using the FFT instead. This is an implementation of the DFT where we avoid repeating certain calculations. This helps speed up the calculation by a lot. However, we are still limited by python which is an easy to use but slow language. The way this is resolved in normal python is by doing the heavy calculations in C in the background. This is the approach libraries like numpy take. We are not working with python, but with micropython. This means that we don't have the same libraries available. While micropython has very few built in libraries we can try to add them manually. However, the development board is still a constrained device with only 8 MB of flash RAM. Numpy needs 64 MB of memory. This means we can't just add numpy to the development board. Numpy is also not a library you can easily dissect and remove the parts you don't need. All of this means we will not be able to use numpy for this project. Instead we will try and implement the DFT ourselves.

The python code for calculating the DFT can be seen in Listing 4.5. The code was written by Chen, 2023.

Listing 4.5: DFT method

```

1
2     def DFT(x):
3         N, yr, yi = len(x), [], []
4         for k in range(N):
5             real, imag = 0, 0
6             for n in range(N):
7                 theta = -k * (2 * math.pi) * (float(n) / N)
8                 real += x[n] * math.cos(theta)
9                 imag += x[n] * math.sin(theta)
10            yr.append(real / N)
11            yi.append(imag / N)
12        return yr, yi
13
14
```

This function returns the real and imaginary part of the DFT. It is an implementation of formula Equation (2.3). As stated before, python is not the fastest programming language out there and this is the same for micropython. This function takes around 60 s to calculate the DFT of a signal with 512 samples. While time is not a great concern for us since we only wake up once every 15 minutes, this will also use a lot of power. While we have not gone further into the power usage, a power analyzer can be used to measure the exact power usage of this function. However, the time it took to calculate was already enough to try and find a better solution. Fortunately we are not the only ones who needed a fast way of calculating the DFT in micropython. There is a library called ulab that is a replacement for numpy in micropython. However, this library is not part of the official pycom firmware. In order to add this we would have to create a custom fork of micropython with the necessary integration. Like most things, this has been done before by someone else. Chen, 2023 has created a custom fork of micropython that has the necessary code for pycom boards and the ulab library. In order to use this firmware we follow the pycom guide on how to flash firmware and select the

custom file instead. The ulab version of numpy is able to calculate the DFT in 0.02 s. This is a huge improvement over the 60 s it took before. This means we can now calculate the DFT in a reasonable amount of time. The code for calculating the DFT using ulab can be seen in Listing 4.6.

Listing 4.6: ulab DFT method

```

1
2     def dft_ulab(arr):
3         c = ulab.array(arr)
4         yr, yi = ulab.fft.fft(c)
5         y = []
6         for i in range(len(yr)):
7             y.append(complex(abs(yr[i]), abs(yi[i])))
8         return y
9
10

```

Even though we are now able to calculate the DFT in a reasonable amount of time, we still have a problem. As stated in Section 2.4, LoRa does not have a great throughput. It is more meant for small packets over long distance. In this case, with 512 samples, the DFT would require us to send 1024 floating point numbers. every number is a total of 24 bytes. We can reduce this number by combining yr and yi in a complex number y. a complex number only takes up 32 bytes. This still means we need to send a total of $512 \cdot 32$ bytes which is 16384 bytes. As stated in Section 2.4, LoRa can only send for 36 s per hour which means we need a data rate of at least $16384 \cdot 8/36 = 3640$ bits/s. While this is a relatively big number for LoRa it can still be done. It will however limit the amount of times we can send per hour. We planned on measuring every 15 minutes but this would require us to have a data rate of $3640 \cdot 4 = 14560$ bits/s which is not possible with LoRa. This means we need to find a more efficient way of transmitting data.

Another, simpler option would be to just send the originally measured data, it is an arr of 512 integers. While this will result in a lower amount of data, it comes with some big privacy issues as you are essentially building a network that is constantly recording everything that happens and transmitting it. This is not something we want to do and will probably bring some legal issues with it.

4.3.3 CALIBRATION

Now that we can calculate the theoretical dB and DFT values, we need to make sure they are correct. To do this we use the setup in fig. 4.3

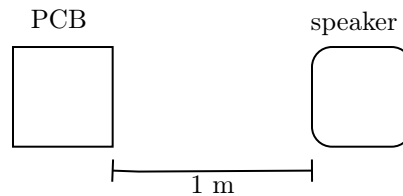


Figure 4.3: Calibration setup

Equation (4.10) is only correct for a perfect microphone. In order to be certain we get accurate dB SPL measurements we need to calibrate the system. This can be done by using a sound source with a known loudness at a specific frequency and comparing that to what we measure.

In this case we used a bose soundlink micro speaker which has been analyzed by Panoryios et al., 2023. According to them, this speaker has a peak loudness of 78.6 dB. We then connect

it to a phone and use an app to generate a sine wave of 1 kHz. We then place the speaker at 1 m of the microphone and measure the output. In order to have a more accurate view of the measurements, we repeat it about 20 times and save the results in a text document. We then calculate the average of these measurements and compare them to what we would expect. As previously said, the speaker we use should produce a peak loudness of 78.6 dB. We then subtract this value from the average of our measurements and find a difference of 14.5 dB. This means we need to subtract 14.5 dB from our measurements to get the correct loudness. We then repeat the previous test several more times in order to make sure we have correctly calibrated our system. After repeating the test 4 more times we find that our result is always within 2 dB of what we would expect. While not being the most accurate, this is more than enough for what we are trying to achieve which is finding where in a city we find the most sound pollution.

This setup can also be used to check the correctness of the FFT calculation. we simply play a sine wave of a certain frequency and compare the output of the FFT to the expected value. We repeat this several times for the same frequency to be sure its consistently correct and then repeat this for different frequencies between 20 Hz and 20 kHz. We find that the FFT gives us an accurate indication of what frequencies are in the signal.

Since we are using 512 samples for a range of 20 kHz, we have a resolution of 39 Hz. This means that we can only detect frequencies that are at least 39 Hz apart. This is not a problem for us since we are only interested in the general frequency spectrum of the sound and not the exact frequencies.

This project is far from finished. There are still several things to be improved or added.

The first issue is that the current calculation for actual data rate is false. It does not account for the fact that we are not working with 8 byte numbers but with 16 bytes. This means that instead of sampling at 38.4 kHz we are actually sampling at 19.2 kHz which is way too low if we want to measure up to 20 kHz. This means we need to add an external MCLK since the internal clock won't go any faster.

Another thing we can improve is the LPF. Currently there is no LPF used since the first order one came with too much noise. We should do more research in order to find a suitable LPF filter.

We should also spend some time analyzing the power consumption of the device since this is an important factor in IoT.

Currently we are measuring 512 samples and using those to calculate the dB and DFT. In order to increase the accuracy we should increase the number of samples. This will however increase the amount of bits that we need to send over LoRa and the current 512 samples are already too much. One way of trying to solve this is figuring out an encoding scheme that will allow us to send less bits while sending more data.

Something we looked over during this project is A-weighting our loudness results. A-weighting is a way of making the dB values resemble what we would actually hear since the human hearing is not linear. This was not implemented since just like FFT it requires a lot of calculations. Depending on whether or not we can send the entire DFT we might could implement this on the receiver side. Otherwise we will need to find a way to implement this using the ulab library.

Finally there is one more factor we did not discuss in this thesis. The noise inside the system. There are several sources of noise. The datasheet specifies how certain settings will affect the noise generated by the ADC. 2 big factors are the OSR and the GAIN. For both the OSR and the GAIN a higher value will result in less noise. The issue however is that to reach a data rate of 38.4 kHz we need to select an OSR of 32 which is the lowest possible value. Together with a GAIN of x8 this will result in a noise RMS level of 17 μ V. This is small enough to not have a major influence on the measurements but is still something worth looking at.

- Babola, T. (Dec. 10, 2020). *Answer to "How to Convert Volts to dB SPL"*. Electrical Engineering Stack Exchange. URL: <https://electronics.stackexchange.com/a/536642> (visited on 06/07/2023).
- Chen, D. (2023). *Python Implementation of DFT and IDFT*. URL: <https://gist.github.com/dannvix/12c26919b6e182afb1f724b051e1ad7a> (visited on 06/10/2023).
- Colistete, C. J. (Aug. 7, 2020). *Pycom · Master · Roberto Colistete Junior / MicroPython Firmwares · GitLab*. GitLab. URL: <https://gitlab.com/rcolistete/micropython-firmwares/-/tree/master/Pycom> (visited on 06/10/2023).
- EXPLAINED, M. S. (2016). *MICROPHONE SPECIFICATIONS EXPLAINED*. URL: <https://invensense.tdk.com/download-pdf/an-1112-microphone-specifications-explained/>.
- Fox, A. (2022). *What Is Microphone Sensitivity? An in-Depth Description*. URL: <https://mynewmicrophone.com/microphone-sensitivity/> (visited on 04/26/2022).
- ‘ICS-40300’ (2023). In: (). URL: https://invensense.tdk.com/wp-content/uploads/2019/02/DS-ICS-40300-00-v1.3.pdf?ref_disty=digikey (visited on 06/05/2023).
- Kordahl, D. (May 17, 2021). *Guessing With Physics*. 3 Quarks Daily. URL: <https://3quarksdaily.com/3quarksdaily/2021/05/guessing-with-physics.html> (visited on 06/11/2023).
- ‘MCP3461’ (2020). In: *MEMS versus ECM (2023). MEMS versus ECM: Comparing Microphone Technologies*. Digi-Key Electronics. URL: <https://www.digikey.be/en/articles/mems-vs-ecm-comparing-microphone-technologies> (visited on 06/06/2023).
- Panoryios, j., Fischer, B., and Khong, Y. (Jan. 19, 2023). *Bose SoundLink Micro Review*. RTINGS.com. URL: <https://www.rtings.com/speaker/reviews/bose/soundlink-micro> (visited on 08/16/2023).
- Peris, E. (2020). ‘Noise Pollution Is a Major Problem, Both for Human Health and the Environment’. In: URL: <https://www.eea.europa.eu/articles/noise-pollution-is-a-major#:~:text=Many%20people%20don%E2%80%99t%20realise%20noise%20pollution%20is%20an,related%20to%20quality%20of%20life%20and%20mental%20health>. (visited on 01/06/2023).
- Smith, S. W. (1999). ‘The Discrete Fourier Transform’. In: *The Scientist & Engineer’s Guide to Digital Signal Processing*. ISBN: ISBN 0-9660176-7-6, ISBN 0-9660176-4-1, ISBN 0-9660176-6-8. URL: https://www.analog.com/en/education/education-library/scientist_engineers_guide.html.
- What Are LoRa and LoRaWAN?* (2023). The Things Network. URL: <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> (visited on 08/17/2023).
- What Noises Cause Hearing Loss?* (Nov. 8, 2022). *What Noises Cause Hearing Loss? | NCEH | CDC*. URL: https://www.cdc.gov/nceh/hearing_loss/what_noises_cause_hearing_loss.html (visited on 06/06/2023).
- Wilczek, J. (2019). *What Is Aliasing? What Causes It? How to Avoid It? - WolfSound*. URL: <https://thewolfound.com/what-is-aliasing-what-causes-it-how-to-avoid-it/> (visited on 06/09/2023).